05. Rust 코드 구현 분석

개요

World Chain은 Rust 언어로 구현되어 있으며, reth(Rust Ethereum) 클라이언트를 기반으로 구축되었습니다. 이 문서는 World Chain의 Rust 코드 구현을 체계적으로 분석하여 전체 crates 구조, PBH 관련 핵심 모듈, 그리고 검증 로직의 구현 방식을 상세히 설명합니다.

5.1 전체 Crates 구조 분석

Workspace 구성

World Chain은 Cargo workspace를 사용하여 여러 crates를 체계적으로 관리합니다. 전체 workspace는 다음과 같이 구성되어 있습니다:

```
[workspace]

resolver = "2"

members = [

"crates/toolkit",

"crates/world/bin",

"crates/world/flashblocks",

"crates/world/node",

"crates/world/payload",

"crates/world/rpc",

"crates/world/pbh",
```

```
"crates/world/pool",
"crates/world/test-utils",
"crates/world/chainspec",
"crates/tests/sepolia",
"crates/tests/devnet",
]
```

핵심 Crates 역할 및 책임

1. world-chain-builder (bin)

- **역할**: World Chain의 메인 실행 파일
- **책임**:
- 노드 초기화 및 실행
- CLI 인터페이스 제공
- Basic 모드와 Flashblocks 모드 지원
- RPC 모듈 확장 및 설정

```
// main.rs의 핵심 구조
fn main() {
let node_config = WorldChainNodeConfig {
args: args.clone(),
da_config: OpDAConfig::default(),
};
match node_context {
```

```
NodeContextType::Basic => {
let  node = WorldChainNode::<BasicContext>::new(node_config);
// Basic 모드 실행
}
NodeContextType::Flashblocks => {
let  node = WorldChainNode::<FlashblocksContext>::new(node_config);
// Flashblocks 모드 실행
}
}
}
```

2. world-chain-builder-node

- **역할**: 노드 구성 및 관리
- **책임**:
- 노드 컨텍스트 관리 (Basic/Flashblocks)
- 페이로드 빌더 통합
- 트랜잭션 풀 관리
- Engine API 처리

3. world-chain-builder-pbh

- **역할**: PBH(Priority Blockspace for Humans) 핵심 로직
- **책임**:
- PBH 페이로드 구조 정의

- Semaphore 증명 검증
- External nullifier 관리
- 날짜 마커 처리

4. world-chain-builder-pool

- **역할**: 트랜잭션 풀 관리
- **책임**:
- PBH 트랜잭션 검증
- 트랜잭션 우선순위 관리
- Nullifier hash 중복 검사
- 메모리 풀 최적화

5. world-chain-builder-payload

- **역할**: 블록 페이로드 구성
- **책임**:
- PBH 트랜잭션 포함 로직
- 블록 빌더 통합
- 가스 한도 관리
- 트랜잭션 정렬

6. world-chain-builder-rpc

- **역할**: RPC 인터페이스 확장
- **책임**:
- World Chain 특화 RPC 메서드
- 시퀀서 클라이언트 통합

- PBH 관련 API 제공

Reth 기반 구현 방식

World Chain은 reth 클라이언트를 기반으로 구축되어 다음과 같은 이점을 제공합니다:

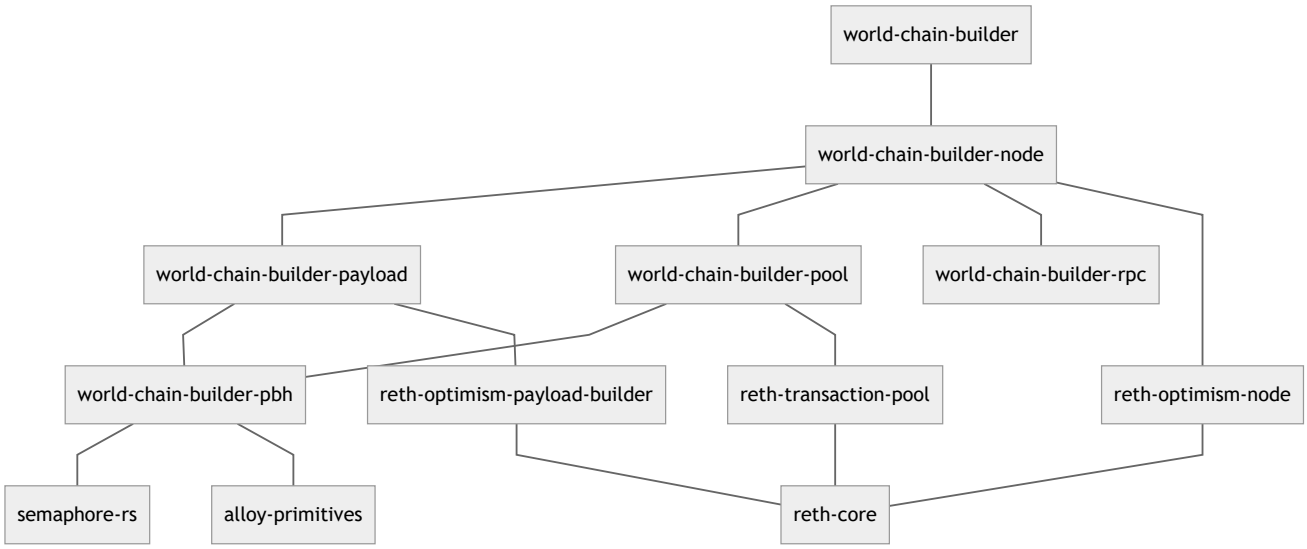
의존성 구조

```
# 핵심 reth 의존성

reth = { git = "https://github.com/paradigmxyz/reth", rev = "6daf5fc" }

reth-optimism-node = { git = "https://github.com/paradigmxyz/reth", rev = "6daf5

reth-optimism-payload-builder = { git = "https://github.com/paradigmxyz/reth", r
```

확장 방식

1. **노드 확장**: `reth-optimism-node` 를 기반으로 World Chain 특화 기능 추가
2. **페이로드 빌더 확장**: `reth-optimism-payload-builder` 를 확장하여 PBH 로직 통합
3. **트랜잭션 풀 확장**: `reth-transaction-pool` 을 확장하여 PBH 검증 추가

아키텍처 다이어그램



외부 의존성

Semaphore 관련

```
semaphore-rs = { version = "0.3.1", features = ["depth_30"] }

semaphore-rs-tree = "0.3.1"

semaphore-rs-proof = "0.3.1"
```

Alloy 생태계

```
alloy-primitives = { version = "1.3.0", features = ["rand", "arbitrary"] }

alloy-rlp = { version = "0.3.10" }

alloy-consensus = { version = "1.0.23" }
```

Rollup Boost 통합

```
rollup-boost = { git = "https://github.com/flashbots/rollup-boost" }

flashblocks-p2p = { git = "https://github.com/flashbots/rollup-boost" }
```

이러한 구조를 통해 World Chain은 기존 Ethereum 클라이언트의 안정성과 성능을 유지하면서도 PBH라는 혁신적인 기능을 효과적으로 통합할 수 있습니다.## 5.2 PBH 관

련 핵심 모듈 분석

world-chain-builder-pbh Crate 상세 분석

PBH(Priority Blockspace for Humans) crate는 World Chain의 핵심 기능을 구현하는 모듈로, Semaphore 영지식 증명 시스템을 기반으로 한 사용자 검증 메커니즘을 제공합니다.

핵심 구조체 및 타입

PBHPayload 구조체

```
#[derive(Default, Clone, Debug, RlpEncodable, RlpDecodable, PartialEq, Eq)]

pub struct PBHPayload {

/// 접두사, 날짜 마커, PBH nonce를 포함하는 문자열

pub external_nullifier: ExternalNullifier,

/// 이전에 사용된 PBH 트랜잭션을 추적하는 nullifier hash

pub nullifier_hash: Field,

/// 이 증명이 생성된 머클 트리의 루트

pub root: Field,

/// 발신자가 orb 검증된 사용자 집합에 포함되어 있음을 증명하는 Semaphore 증명

pub proof: Proof,
```

```
}
```

ExternalNullifier 구조체

```
#[derive(Builder, Debug, Default, Clone, Copy, PartialEq, Eq)]
pub struct ExternalNullifier {
pub version: Prefix, // 버전 정보 (현재 V1)
pub year: u16, // 연도
pub month: u8, // 월
pub nonce: u16, // 논스 (월별 트랜잭션 제한용)
}
```

검증 로직 구현

1. 통합 검증 메서드

```
pub fn validate(
&self,
signal: U256,
valid_roots: &[Field],
pbh_nonce_limit: u16,
) -> Result<(), PBHValidationError> {
// 1. 머클 루트 검증
self.validate_root(valid_roots)?;
// 2. External nullifier 검증 (날짜 및 논스)
```

```
let date = chrono::Utc::now();
self.validate_external_nullifier(date, pbh_nonce_limit)?;
// 3. Semaphore 증명 검증
let flat = self.proof.0.flatten();
let proof = if (flat[4] | flat[5] | flat[6] | flat[7]).is_zero() {
// 압축된 증명 처리
let compressed_flat = [flat[0], flat[1], flat[2], flat[3]];
let compressed_proof =
semaphore_rs_proof::compression::CompressedProof::from_flat(compressed_flat);
&semaphore_rs_proof::compression::decompress_proof(compressed_proof)
.ok_or(PBHValidationError::InvalidProof)?
} else {
&self.proof.0
};
// Semaphore 증명 검증 실행
if verify_proof(
self.root,
self.nullifier_hash,
signal,
EncodedExternalNullifier::from(self.external_nullifier).0,
proof,
TREE_DEPTH,
)? {
```

```
Ok(())
} else {
Err(PBHValidationError::InvalidProof)
}
}
```

2. 인코딩 방식

External nullifier는 다음과 같은 비트 구조로 인코딩됩니다:

- Bits 48-255: 빈 공간
- Bits 40-47: 연도
- Bits 24-39: 월
- Bits 8-23: 논스
- Bits 0-7: 버전

world-chain-builder-pool Crate 트랜잭션 풀 구현 분석

트랜잭션 풀은 PBH 트랜잭션의 검증과 관리를 담당하는 핵심 컴포넌트입니다.

WorldChainTransactionValidator 구조

```
pub struct WorldChainTransactionValidator<Client, Tx> {
/// 내부 트랜잭션 검증기 (OP Stack 기반)
inner: OpTransactionValidator<Client, Tx>,
/// World ID 증명의 유효한 루트를 검증하는 검증기
root_validator: WorldChainRootValidator<Client>,
/// 단일 World ID가 한 달에 실행할 수 있는 최대 PBH 트랜잭션 수
```

```
max_pbh_nonce: Arc<AtomicU16>,

/// 단일 PBH 트랜잭션이 소비할 수 있는 최대 가스량

max_pbh_gas_limit: Arc<AtomicU64>,

/// 모든 PBH 트랜잭션의 엔트리포인트 주소

pbh_entrypoint: Address,

/// World ID PBH 서명 집계기 주소

pbh_signature_aggregator: Address,

}
```

PBH 번들 검증 프로세스

```
pub  async  fn  validate_pbh_bundle(

&self,

origin: TransactionOrigin,

tx: Tx,

) -> TransactionValidationOutcome<Tx> {

// 1. 기본 OP 트랜잭션 검증

let  mut  tx_outcome = self.inner.validate_one(origin, tx.clone()).await;

if !tx_outcome.is_valid() {

return  tx_outcome;

}


// 2. 콜데이터 디코딩 및 서명 집계기 검증
```

```
let Ok(calldata) = IPBHEntryPoint::handleAggregatedOpsCall::abi_decode(tx.input(

return WorldChainPoolTransactionError::from(PBHValidationError::InvalidCalldata)

.to_outcome(tx);

};



// 3. 모든 UserOp가 올바른 PBH 서명 집계기를 지정하는지 확인

if !calldata._0.iter().all(|aggregator|

aggregator.aggregator == self.pbh_signature_aggregator) {

return WorldChainPoolTransactionError::from(

PBHValidationError::InvalidSignatureAggregator,

).to_outcome(tx);

}



// 4. 각 UserOp와 연관된 모든 증명 검증

let  mut  aggregated_payloads = vec![];

let  mut  seen_nullifier_hashes = HashSet::new();



for  aggregated_ops  in  calldata._0 {

// PBH 페이로드 디코딩

let  pbh_payloads = match <Vec<PBHPayload>>::abi_decode(buff) {

Ok(pbh_payloads) => pbh_payloads,

Err(_) => return  error_outcome,

};
```

```
// 병렬 검증 실행
let payloads: Vec<PbhPayload> = pbh_payloads
.into_par_iter()
.zip(aggregated_ops.userOps)
.map(|(payload, op)| {
let signal = crate::eip4337::hash_user_op(&op);
let payload = PbhPayload::try_from(payload)?;
payload.validate(
signal,
&valid_roots,
self.max_pbh_nonce.load(Ordering::Relaxed),
)?;
Ok(payload)
})
.collect::<Result<Vec<PbhPayload>, WorldChainPoolTransactionError>>()?;


// 5. 중복 nullifier hash 검사
for payload in &payloads {
if !seen_nullifier_hashes.insert(payload.nullifier_hash) {
return WorldChainPoolTransactionError::from(
PBHValidationError::DuplicateNullifierHash,
).to_outcome(tx);
```

```
}
}
}
}
```

world-chain-builder-payload Crate 페이로드 빌더 분석

페이로드 빌더는 PBH 트랜잭션을 포함한 블록을 구성하는 역할을 담당합니다.

WorldChainPayloadBuilder 구조

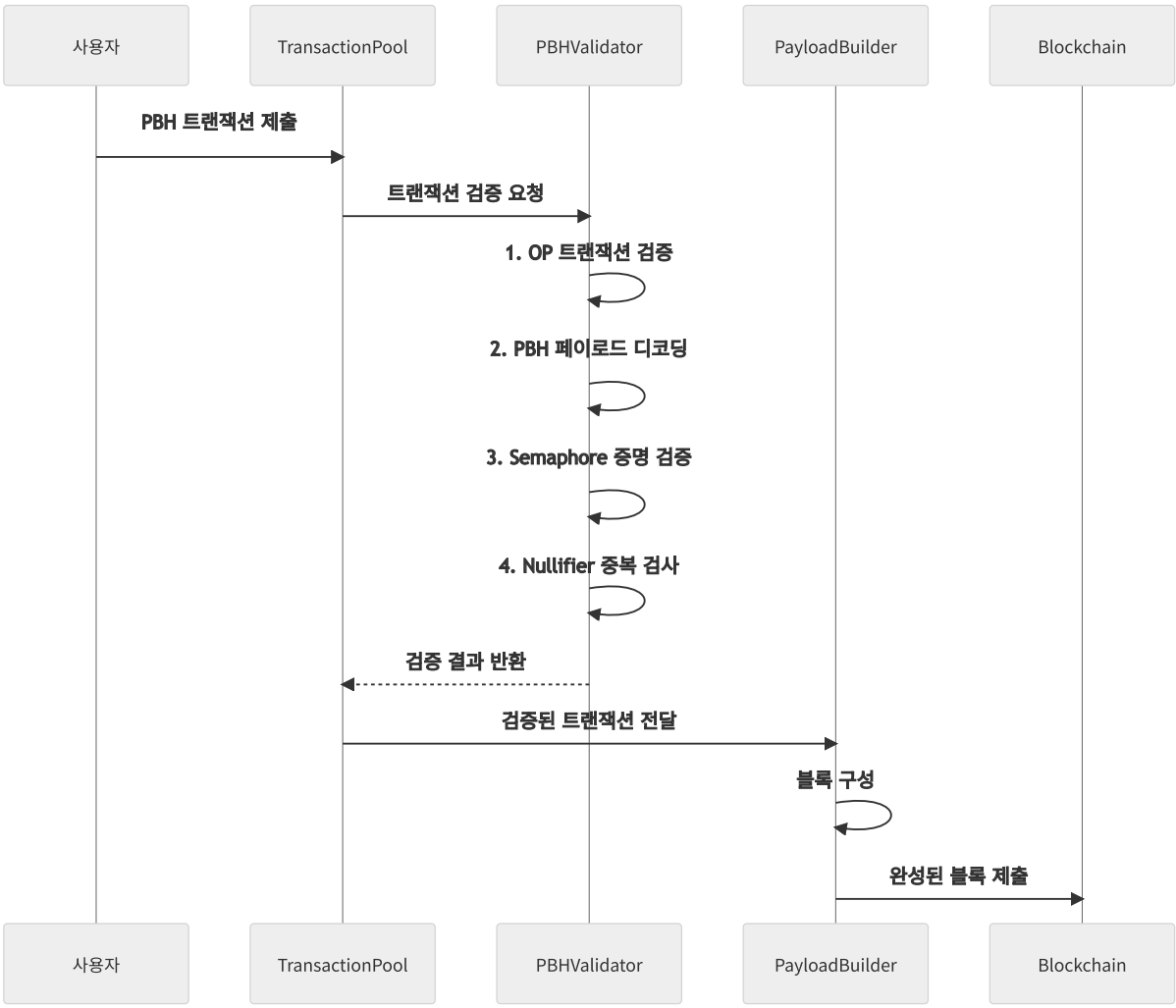
```
pub struct WorldChainPayloadBuilder<Client, S, Txs = ()> {
/// 내부 OP 페이로드 빌더
pub inner: OpPayloadBuilder<WorldChainTransactionPool<Client, S>, Client, OpEvm
/// 검증된 블록스페이스 용량 (백분율)
pub verified_blockspace_capacity: u8,
/// PBH 엔트리포인트 주소
pub pbh_entry_point: Address,
/// PBH 서명 집계기 주소
pub pbh_signature_aggregator: Address,
/// 빌더 개인키
pub builder_private_key: PrivateKeySigner,
}
```

블록 구성 프로세스

```
pub fn build<Pool, Client>(
self,
db: impl Database<Error = ProviderError>,
state_provider: impl StateProvider,
ctx: WorldChainPayloadBuilderCtx<Client, Pool>,
) -> Result<BuildOutcomeKind<OpBuiltPayload<OpPrimitives>>, PayloadBuilderError>
// 1. 상태 및 블록 빌더 준비
let mut state = State::builder()
.with_database(db)
.with_bundle_update()
.build();
let mut builder = PayloadBuilderCtx::block_builder(&ctx, &mut state)?;
// 2. 실행 전 변경사항 적용
builder.apply_pre_execution_changes()?;
// 3. 시퀀서 트랜잭션 실행
let mut info = op_ctx.execute_sequencer_transactions(&mut builder)?;
// 4. 메모리 풀 트랜잭션 실행 (허용된 경우)
if !op_ctx.attributes().no_tx_pool {
let best_txs = best(op_ctx.best_transaction_attributes(builder.evm_mut().block(
if ctx.execute_best_transactions(&mut info, &mut builder, best_txs, gas_limit
.is_none() {
return Ok(BuildOutcomeKind::Cancelled);
```

```
}

// 새 페이로드가 더 가치 있는지 확인

if !ctx.inner.is_better_payload(info.total_fees) {

return Ok(BuildOutcomeKind::Aborted { fees: info.total_fees });

}

}

// 5. 블록 완성: 모든 루트 계산 (트랜잭션, 상태)

let BlockBuilderOutcome {

execution_result,

hashed_state,

trie_updates,

block,

} = builder.finish(state_provider)?;

// 6. 최종 페이로드 생성

let  payload = OpBuiltPayload::new(

op_ctx.payload_id(),

sealed_block,

info.total_fees,

Some(executed),

);

Ok(BuildOutcomeKind::Better { payload })

}
```

아키텍처 다이어그램



이러한 구조를 통해 World Chain은 기존 Ethereum 트랜잭션과 PBH 트랜잭션을 모두 효율적으로 처리하면서, 인간 사용자에게 우선순위를 부여하는 혁신적인 메커니즘을 구현합니다.#

5.3 검증 로직 구현 분석

PBH 트랜잭션 검증 알고리즘

World Chain의 PBH 트랜잭션 검증은 다층적인 검증 시스템으로 구성되어 있으며, 각 단계에서 서로 다른 측면의 유효성을 확인합니다.

1. 다단계 검증 프로세스

1단계: 기본 트랜잭션 검증

```
// OpTransactionValidator를 통한 기본 검증
let mut tx_outcome = self.inner.validate_one(origin, tx.clone()).await;
if !tx_outcome.is_valid() {
return tx_outcome;
}
```

2단계: PBH 엔트리포인트 확인

```
// PBH 엔트리포인트로의 트랜잭션인지 확인
if transaction.to().unwrap_or_default() != self.pbh_entrypoint {
return self.inner.validate_one(origin, transaction.clone()).await;
}

// 가스 한도 검증
if tx.gas_limit() > self.max_pbh_gas_limit.load(Ordering::Relaxed) {
return WorldChainPoolTransactionError::from(PBHValidationError::PbhGasLimitExcee
```

```
.to_outcome(tx);
}
```

3단계: 함수 시그니처 및 콜데이터 검증

```
let function_signature: [u8; 4] = tx
.input()
.get(..4)
.and_then(|bytes| bytes.try_into().ok())
.unwrap_or_default();

match function_signature {
IPBHEntryPoint::handleAggregatedOpsCall::SELECTOR => {
self.validate_pbh_bundle(origin, tx).await
}
_ => self.inner.validate_one(origin, tx.clone()).await,
}
```

2. Semaphore 증명 검증 구현

증명 압축 처리

```
let flat = self.proof.0.flatten();
let proof = if (flat[4] | flat[5] | flat[6] | flat[7]).is_zero() {
// 압축된 증명 처리
```

```
let compressed_flat = [flat[0], flat[1], flat[2], flat[3]];
let compressed_proof =
semaphore_rs_proof::compression::CompressedProof::from_flat(compressed_flat);
&semaphore_rs_proof::compression::decompress_proof(compressed_proof)
.ok_or(PBHValidationError::InvalidProof)?
} else {
// 비압축 증명 사용
&self.proof.0
};
```

Semaphore 프로토콜 검증

```
if verify_proof(
self.root, // 머클 트리 루트
self.nullifier_hash, // Nullifier hash
signal, // UserOperation hash
EncodedExternalNullifier::from(self.external_nullifier).0, // External nullifier
proof, // 영지식 증명
TREE_DEPTH, // 트리 깊이 (30)
)? {
Ok(())
} else {
Err(PBHValidationError::InvalidProof)
```

```
}
```

3. Nullifier Hash 중복 검사 메커니즘

트랜잭션 내 중복 검사

```
let mut seen_nullifier_hashes = HashSet::new();

for payload in &payloads {
if !seen_nullifier_hashes.insert(payload.nullifier_hash) {
return WorldChainPoolTransactionError::from(
PBHValidationError::DuplicateNullifierHash,
).to_outcome(tx);
}
}
```

UserOperation Signal 생성

```
pub fn hash_user_op(user_op: &PackedUserOperation) -> Field {
let hash = SolValue::abi_encode_packed(&(
&user_op.sender,
&user_op.nonce,
&user_op.callData
));
hash_to_field(hash.as_slice())
```

```
}
```

World ID 루트 관리 시스템

RootProvider 구현

루트 캐싱 및 만료 관리

```
pub struct RootProvider<Client> {
/// WorldID 컨트랙트 주소
world_id: Address,
/// 데이터베이스 상태 접근용 클라이언트
client: Client,
/// 블록 타임스탬프별로 인덱싱된 유효한 루트 맵
valid_roots: BTreeMap<u64, Field>,
/// 최신 유효 루트의 타임스탬프
latest_valid_timestamp: u64,
/// 최신 루트
latest_root: Field,
}
```

루트 만료 정책

```
/// 루트 만료 기간: 1주일
pub const ROOT_EXPIRATION_WINDOW: u64 = 60 * 60 * 24 * 7;
```

```
fn prune_invalid(&mut self) {
if self.latest_valid_timestamp > ROOT_EXPIRATION_WINDOW {
self.valid_roots.retain(|timestamp, root| {
*timestamp >= self.latest_valid_timestamp - ROOT_EXPIRATION_WINDOW
|| *root == self.latest_root // 최신 루트는 항상 유지
});
};
}
```

새 블록 처리

```
fn on_new_block<B>(&mut self, block: &SealedBlock<B>) -> Result<(), WorldChain
let state = self.client.state_by_block_hash(block.hash())?;
let root = state.storage(self.world_id, LATEST_ROOT_SLOT.into())?;
self.latest_valid_timestamp = block.timestamp();
if let Some(root) = root {
self.valid_roots.insert(block.timestamp(), root);
}
self.prune_invalid();
Ok(())
}
```

트랜잭션 우선순위 시스템

WorldChainOrdering 구현

우선순위 결정 로직

```
#[derive(Debug, Default, Clone, Ord, PartialOrd, Eq, PartialEq)]
pub struct WorldChainPriority {
is_pbh: bool, // PBH 트랜잭션 여부 (최우선)
effective_tip_per_gas: Option<U256>, // 가스당 팁 (차순위)
}

fn priority(&self, transaction: &Self::Transaction, base_fee: u64) -> Priority<
let effective_tip_per_gas = transaction.effective_tip_per_gas(base_fee).map(U25

Some(WorldChainPriority {
is_pbh: transaction.pbh_payload().is_some(),
effective_tip_per_gas,
}).into()
}
```

이 구조에서 PBH 트랜잭션은 가스 가격에 관계없이 항상 일반 트랜잭션보다 높은 우선순위를 가집니다.

검증 오류 처리

오류 타입 정의

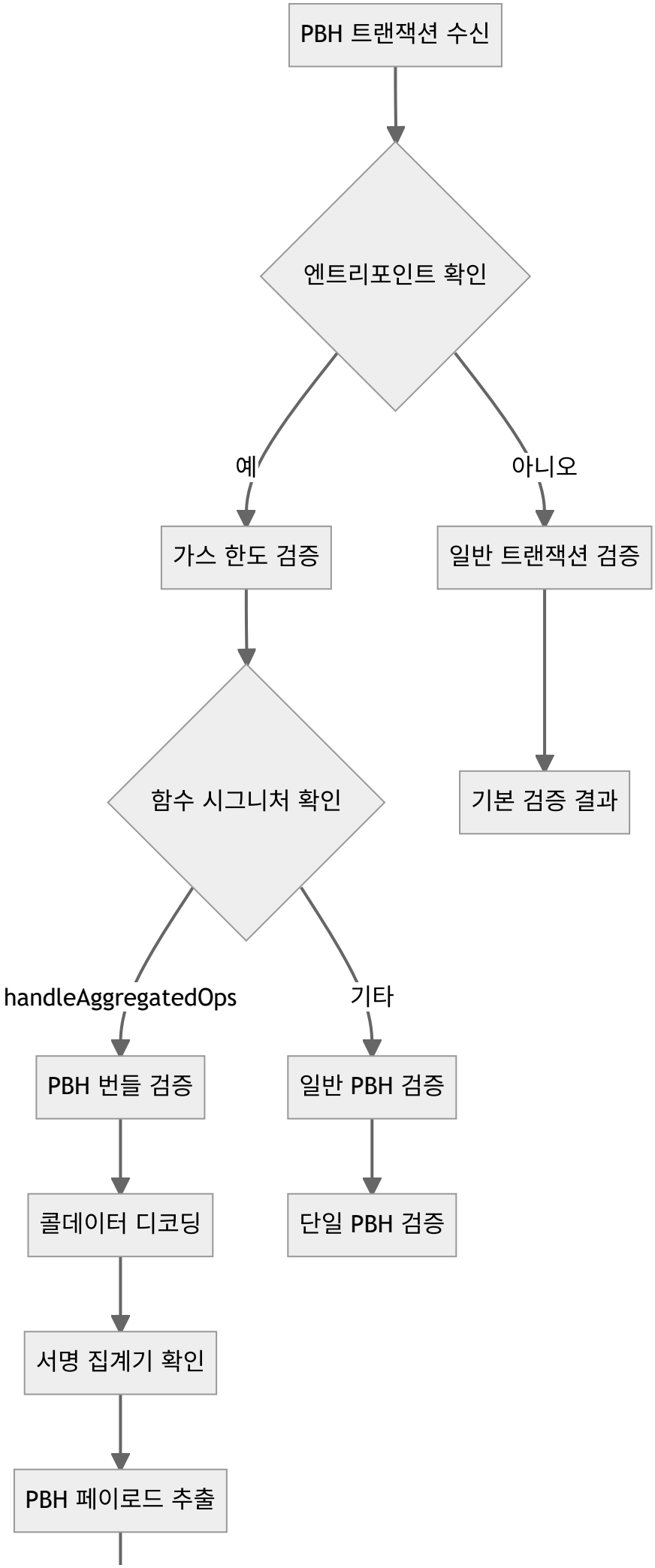
```
#[derive(Error, Debug)]
```

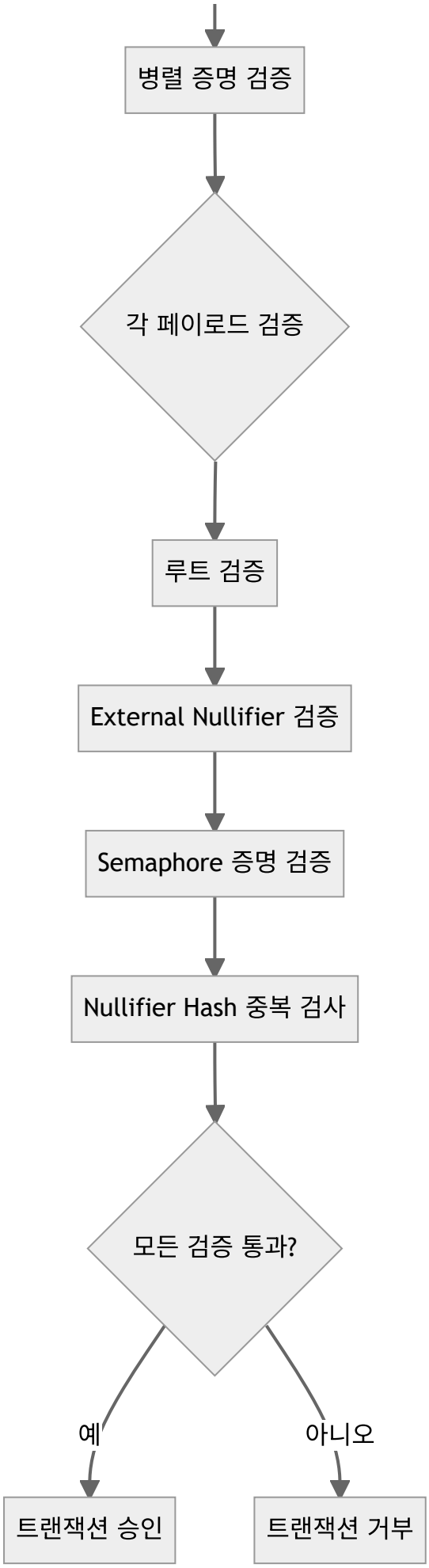
```rust
pub  enum PBHValidationError {
#[error("Invalid root")]
InvalidRoot,
#[error("Invalid external nullifier period")]
InvalidExternalNullifierPeriod,
#[error("Invalid external nullifier nonce")]
InvalidExternalNullifierNonce,
#[error("Invalid proof")]
InvalidProof,
#[error("Invalid calldata encoding")]
InvalidCalldata,
#[error("Missing PBH Payload")]
MissingPbhPayload,
#[error("InvalidSignatureAggregator")]
InvalidSignatureAggregator,
#[error("PBH gas limit exceeded")]
PbhGasLimitExceeded,
#[error("Duplicate nullifier hash")]
DuplicateNullifierHash,
}
```

병렬 검증 최적화

```
let  payloads: Vec<PbhPayload> = pbh_payloads

.into_par_iter() // Rayon을 사용한 병렬 처리

.zip(aggregated_ops.userOps)

.map(|(payload, op)| {

let  signal = crate::eip4337::hash_user_op(&op);

let  payload = PbhPayload::try_from(payload)?;

payload.validate(signal, &valid_roots, self.max_pbh_nonce.load(Ordering::Relaxed

Ok(payload)

})

.collect::<Result<Vec<PbhPayload>, WorldChainPoolTransactionError>>()?;
```

검증 플로우 다이어그램


PBH 트랜잭션 수신
엔트리포인트 확인
예
아니오
가스 한도 검증
일반 트랜잭션 검증
함수 시그니처 확인
기본 검증 결과
handleAggregatedOps
기타
PBH 번들 검증
일반 PBH 검증
콜데이터 디코딩
단일 PBH 검증
서명 집계기 확인
PBH 페이로드 추출



이러한 다층적 검증 시스템을 통해 World Chain은 PBH 트랜잭션의 무결성과 보안을 보장하면서도,

높은 처리량을 유지할 수 있습니다. 특히 병렬 처리와 효율적인 캐싱 메커니즘을 통해 검증 성능을 최적화하고 있습니다.